Predicting KV Cache Transfer Outcomes at Prefill

Wolfie, Dashiell

Background: LLM Routing

LLM routers predict which model is best suited for a query (usually to minimize cost subject to a quality constraint). There are three main strategies:

Prompt embeddings	**Model output**	**Prefill activations**
Pro: predict model early *Con:* prompt may not inform how difficult the task is	*Pro:* cascade model when the previous one struggles *Con:* predict model late, re-prefill needed	*Pro:* predict model early-ish, better representation of task *Con:* re-prefill needed

In general, we want fast, accurate predictions with low cost for misrouting.

Background: LLM Routing

LLM routers predict which model is best suited for a query (usually to minimize cost subject to a quality constraint). There are three main strategies:

Prompt embeddings	**Model output**	**Prefill activations**
Pro: predict model early *Con:* prompt may not inform how difficult the task is	*Pro:* cascade model when the previous one struggles *Con:* predict model late, re-prefill needed	*Pro:* predict model early-ish, better representation of task *Con:* re-prefill needed

In general, we want fast, accurate predictions with low cost for misrouting.

Background: LLM Routing

Predicting on prefill activations is less common but seems to work well.

Main limitation: may need to run prefill more than once.

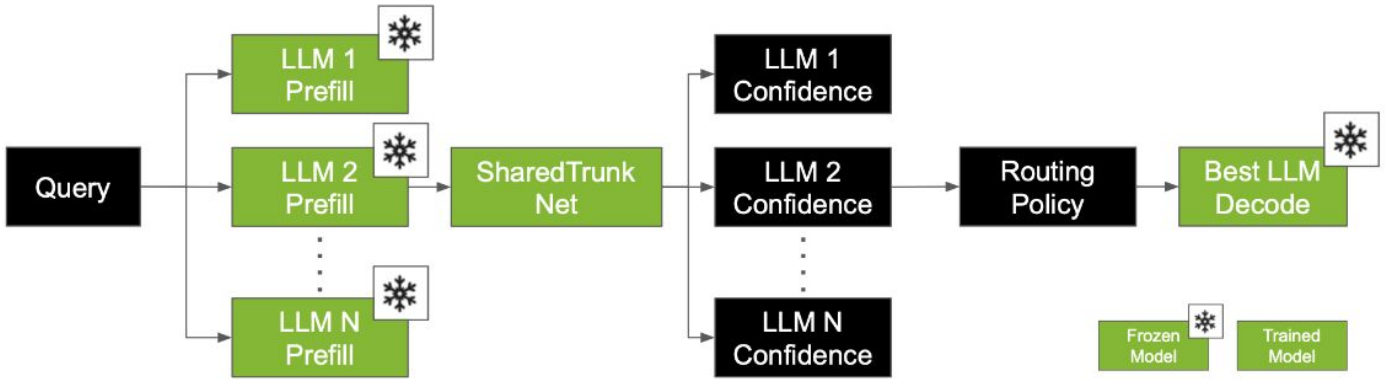


Figure from Varshney et al., "Rethinking Routing with Prefill Activations," arXiv:arXiv:2603.20895 (2026).

Background: KV Cache Transfer

KV caches seem to be linearly related. This lets us convert KV caches between models (usually in the same family, but not necessarily).

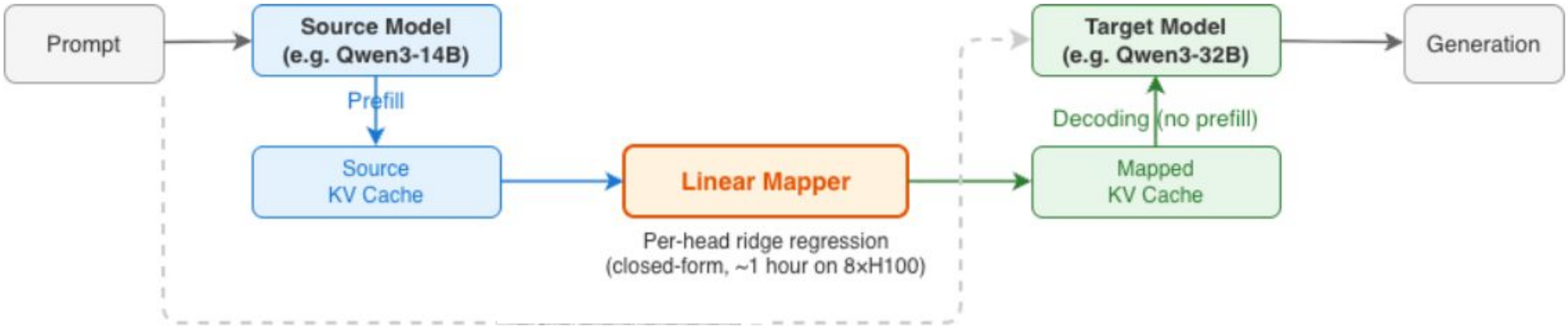


Figure from Heo et al., "Cross-Model KV Cache Transfer in LLM Families," arXiv:2608.03893 (2026).

Background: KV Cache Transfer

Transferring KV caches works reasonably well.

Main limitation: only basic evaluation, hard to predict when KV transfer will work.

Table 3: Ridge vs. MLP HellaSwag retention on four Qwen3 and Ministral 3 pairs.

Pair	Ridge	MLP	Δ
Qwen3 14B→32B	97.6%	97.3%	−0.3 pp
Ministral 3B→8B	93.3%	91.8%	−1.5 pp
Ministral 3B→14B	**68.0%**	**92.3%**	**+24.3 pp**
Ministral 8B→14B	**58.7%**	**95.5%**	**+36.8 pp**

Figure from Heo et al., "Cross-Model KV Cache Transfer in LLM Families," arXiv:2608.03893 (2026).

Background: Disaggregated Prefill

Processing input tokens (prefill) and output tokens (decode) are very different tasks. The former is compute-bound, the latter is memory-bound.

It makes sense to serve these on different machines.

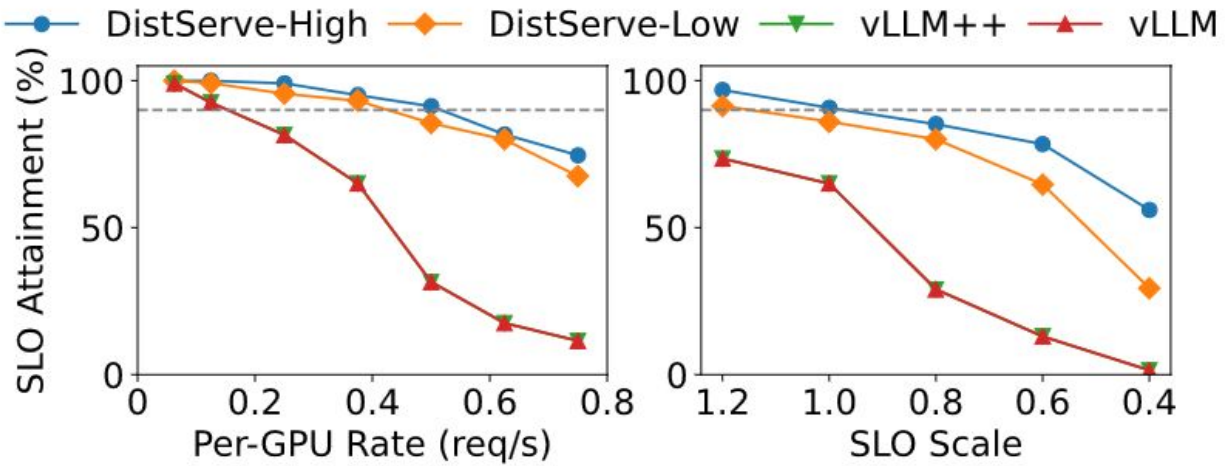


Figure from Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," arXiv:2401.09670 (2024).

Proposal

What if we route our request after prefill using the model's activations, and use KV cache transfer to avoid recomputing the KV?

This is a fairly obvious idea, but the novelty is in characterizing when KV transfer will perform well enough and when re-prefilling is necessary.

We'll need to reevaluate KV transfer and train a predictor on activations.

Details

More specifically, it’s nontrivial how to:

1. Implement this at a systems level
2. Evaluate successful KV cache transfer
3. Evaluate a good routing mechanism
4. Develop a runtime policy based on this
5. Quantify throughput gains

We will investigate each of these questions.

Thank you